Build a Full-Stack Palestinian Grocery Marketplace Platform

1. Project Overview

Build a production-ready multi-vendor grocery marketplace platform for Palestine.

The platform allows customers to:

- Select their governorate and city.
- Browse registered supermarkets, malls, grocery stores, and food retailers available in their area.
- Enter any store.
- Browse categories and products.
- Add products to a shopping cart.
- Continue shopping from other stores without losing previously selected products.
- Maintain separate cart sections for each store.
- Place one checkout containing products from multiple stores.
- Track orders and order status.

The platform also allows any eligible supermarket or grocery store in Palestine to register, get verified by the platform administrator, upload products, manage inventory and prices, receive orders, and manage deliveries.

The system must be designed as a scalable marketplace rather than a single-store e-commerce application.

---

2. Target Market

The application is designed specifically for the Palestinian market.

Support:

- Arabic as the primary language.
- Full RTL interface.
- English as an optional secondary language.
- Palestinian governorates and cities.
- Prices in Israeli New Shekel (ILS / ₪).
- Local phone number formats.
- Local delivery addresses.
- Cash on delivery.
- Online payment architecture prepared for future Palestinian payment providers.

- Store-based delivery.

Do not hard-code geographic data. Create database tables for governorates, cities, areas, and delivery zones so the administrator can manage them.

---

3. Recommended Technology Stack

Use the following architecture:

Frontend Mobile

Flutter.

The mobile application must support:

- Android
- iOS
- Arabic RTL
- Responsive layouts
- Clean modern UI
- State management using a maintainable architecture such as Riverpod or Bloc.

Backend

Node.js + NestJS + TypeScript.

Create a RESTful API with:

- Authentication
- Authorization
- Product APIs
- Store APIs
- Cart APIs
- Order APIs
- Delivery APIs
- Admin APIs
- Notification APIs

Database

PostgreSQL.

Use Prisma ORM.

Admin Dashboard

Next.js + TypeScript.

Use a modern component system and responsive dashboard.

Storage

Use a cloud-compatible object storage architecture for:

- Product images
- Store logos
- Store images
- User profile images
- Documents used for store verification

The storage provider must be configurable through environment variables.

---

4. System Architecture

Create the following structure:

palestine-grocery-marketplace/

|

├── apps/

| ├── mobile/

| ├── admin/

| └── api/

|

├── packages/

| ├── shared/

| ├── types/

| ├── validation/

| └── config/

|

├── database/

| ├── prisma/

| └── seed/

|

├── docs/

│

├── docker/

│

├── .env.example

├── docker-compose.yml

├── README.md

└── package.json

Use clean separation between frontend, backend, database, and shared types.

---

5. User Roles

Implement role-based access control.

Roles:

CUSTOMER

Can:

- Register
- Login
- Select location
- Browse stores
- Browse products
- Manage cart
- Place orders
- Track orders
- Review stores/products
- Manage addresses
- View order history

لأدوار المستخدمين ورُموزها التقنية (RBAC Roles) الخاصة بالمطور:

١. **المصنع / تاجر جملة الجملة (MEGA_WHOLESALER / MANUFACTURER)**

٢. **تاجر الجملة (WHOLESALER)**

٣. **التاجر / تاجر التجزئة (MERCHANT / RETAILER)**

٤. **الزبون / المستهلك النهائي (CUSTOMER)**

5. ADMIN

Can:

- Manage users
- Approve/reject stores
- Manage stores
- Manage products
- Manage categories
- Manage governorates
- Manage cities
- Manage delivery zones
- Manage orders
- Manage commissions
- Manage promotions
- Manage reports
- Suspend users/stores
- Configure platform settings

---

6. Geographic System

Create:

Governorate

```
|
└── City
    |
    └── Area
        |
        └── Delivery Zone
```

Example seed data:

- Jerusalem
- Ramallah and Al-Bireh
- Nablus
- Hebron
- Bethlehem
- Jenin
- Tulkarm
- Qalqilya
- Salfit
- Jericho and Al-Aghwar
- Tubas

Do not assume this list is exhaustive. The admin must be able to add/edit/remove geographic entities.

---

7. Customer Registration

Registration fields:

- Full name
- Mobile number
- Email optional
- Password
- Governorate
- City
- Area
- Address

- Location Based GPS.
- Optional location coordinates

Authentication:

- Phone
- Password
- JWT access token
- Refresh token

Prepare architecture for OTP verification.

---

8. Home Screen

Design a modern Arabic RTL grocery marketplace homepage.

Sections:

Header

نابلس

مرحباً، محمد

ماذا تريد أن تشتري؟

Categories

Horizontal category list:

- مواد غذائية
- ألبان وأجبان
- لحوم
- دواجن
- خضار وفواكه
- مشروبات
- مخبوزات
- معلبات
- منظفات
- ورقيات
- مستلزمات الأطفال
- مجمدات

Promotions

Show:

- Today's offers
- Discounts
- Featured stores
- Featured products

Stores Near Customer

Display stores according to:

1. Governorate
2. City
3. Delivery availability
4. Distance if location is available
5. Store status

---

9. Store Marketplace

Create a store discovery page.

Each store card should display:

- Store logo
- Store name
- Cover image
- Rating
- Delivery fee
- Estimated delivery time
- Minimum order
- Open/closed status
- Distance when available
- Promotion badge

Filters:

- Nearest
- Highest rated
- Lowest delivery fee

- Fastest delivery
- Offers

---

10. Store Page

When the customer enters a store:

Display:

[Store Cover]

Store Name

★★★★★ 4.7

Delivery: 10 ₪

Estimated: 30–45 min

Search products...

Then display store categories.

Example:

All

Rice & Grains

Oil

Sugar

Canned Food

Dairy

Drinks

Cleaning

Products should include:

- Image
- Name
- Brand
- Size
- Price

- Discounted price
- Availability
- Add button

---

11. Product Details

Product page:

- Large product image
- Product name
- Brand
- Description
- Size/weight
- Original price
- Discount price
- Availability
- Quantity selector
- Add to cart

Optional:

- Product rating
- Reviews
- Related products

---

12. MULTI-STORE SHOPPING CART

This is one of the most important requirements.

The customer must be able to shop from multiple stores before checkout.

Example:

MY CART

Store A

Rice 5kg 25 ₪

Oil 30 ₪

Sugar 10 ₪

Subtotal 65 ₪

Store B

Milk 15 ₪

Juice 12 ₪

Subtotal 27 ₪

Store C

Detergent 20 ₪

Subtotal 20 ₪

Products Total: 112 ₪

Delivery: 25 ₪

Grand Total: 137 ₪

Never overwrite products when the user changes stores.

Implement:

Cart

├── StoreCart A

│ ├── CartItem

│ ├── CartItem

│ └── CartItem

│

├── StoreCart B

│ ├── CartItem

│ └── CartItem

│

└── StoreCart C

└── CartItem

Each store cart must preserve:

- Store ID
- Products
- Quantities
- Prices at the time of cart update
- Discounts
- Subtotal
- Delivery fee

---

13. CHECKOUT

Create a unified checkout experience.

The customer sees:

Delivery Address

Allow:

- Existing address
- Add new address
- Edit address

Store Orders

Display each store separately.

Example:

Order 1

Al-Nour Market

Subtotal: 80 ₪

Delivery: 10 ₪

Order 2

Palestine Mall

Subtotal: 120 ₪

Delivery: 15 ₪

Payment

Support:

- Cash on delivery
- Online payment placeholder
- Future payment providers

Architecture must allow payment providers to be added later without rewriting checkout.

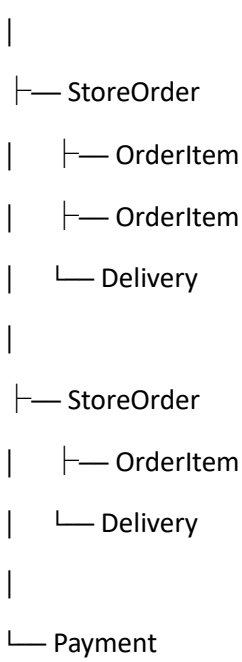
---

14. ORDER MODEL

Create a parent order and child store orders.

Example:

MasterOrder



Each store independently receives and processes its own StoreOrder.

---

15. ORDER STATUS

Customer order statuses:

Pending

Confirmed

Preparing

Ready for Pickup

Out for Delivery

Delivered

Cancelled

Store status updates should automatically update the customer interface.

---

16. STORE REGISTRATION

Create a complete store onboarding flow.

Fields:

- Store name
- Owner name
- Mobile
- Email
- Governorate
- City
- Area
- Address
- Coordinates
- Store description
- Logo
- Cover image
- Opening hours
- Delivery zones
- Delivery fee
- Minimum order
- Estimated preparation time
- Location Based GPS

Verification documents should be supported.

Status:

PENDING_REVIEW

APPROVED

REJECTED

SUSPENDED

Admin approval is required before the store becomes publicly visible.

---

17. STORE DASHBOARD

Create a professional Arabic RTL dashboard.

Main cards:

Today's Orders

Today's Sales

Pending Orders

Products

Low Stock

Pages:

Dashboard

Charts:

- Sales by day
- Orders by day
- Best-selling products

Products

Actions:

- Add
- Edit
- Delete
- Enable/disable
- Change price
- Change inventory

Categories

Manage store categories.

Orders

Table:

Order #

Customer

Items

Total

Payment

Status

Date

Actions

Promotions

Create:

- Percentage discount
- Fixed discount
- Buy X Get Y
- Product promotion
- Category promotion
- Time-limited offer

Store Settings

Manage:

- Logo
- Cover
- Address
- Hours
- Delivery
- Minimum order

---

18. INVENTORY

Every product should have inventory fields.

Example:

Product

SKU

Stock Quantity

Low Stock Threshold

Availability

Automatically mark products as:

IN STOCK

LOW STOCK

OUT OF STOCK

Prevent customers from purchasing unavailable quantities.

---

19. SEARCH

Implement global product search.

Search should support:

- Arabic
- English
- Partial names
- Brand
- Category

Example:

Customer searches:

زيت زيتون

Return products across stores.

Show:

Product

Store

Price

Availability

Delivery

Allow filtering by:

- Price
- Store
- Category
- Rating
- Offers

---

20. PRICE COMPARISON

Create an optional feature:

Customer searches:

زيت زيتون ١ لتر

The system displays:

Store A 30 ₪

Store B 28 ₪

Store C 32 ₪

Allow customer to add a product from the preferred store.

---

21. SMART SHOPPING ASSISTANT

Prepare an AI-ready module.

Customer can type:

أريد شراء مواد غذائية للبيت بميزانية ٣٠٠ شيكل.

The system should eventually generate a suggested basket.

Architecture must isolate AI services so an AI provider can be integrated later.

Do not require an AI API key for the initial MVP.

Create an interface:

ShoppingAssistantService

with a mock implementation for development.

---

22. DELIVERY SYSTEM

Initially allow stores to manage their own delivery.

Create future-ready Driver functionality.

Driver model:

Driver

User

Vehicle

Availability

Current Location

Status

Delivery statuses:

Assigned

Accepted

Picking Up

Picked Up

On The Way

Delivered

Failed

---

23. NOTIFICATIONS

Create notification infrastructure.

Notifications for:

Customer

- Order received
- Order confirmed
- Order preparing
- Order dispatched
- Order delivered
- Order cancelled
- Promotion

Store

- New order
- Order cancelled
- Low inventory

Driver

- New delivery
- Delivery cancelled

Prepare support for:

- Push notifications
- SMS
- Email

---

24. RATINGS AND REVIEWS

Customers can rate stores after completed orders.

Rating:

1–5 stars.

Allow reviews.

Store rating should be calculated automatically.

Prevent customers from reviewing orders that they did not complete.

---

25. ADMIN DASHBOARD

Build a powerful admin dashboard.

Sidebar:

Dashboard

Users

Stores

Store Applications

Products

Categories

Orders

Deliveries

Drivers

Governorates

Cities

Areas

Delivery Zones

Promotions

Reviews

Payments

Commissions

Reports

Settings

Dashboard statistics:

Total Customers

Total Stores

Pending Stores

Total Orders

Today's Orders

Total Revenue

Platform Commission

Active Drivers

---

26. ADMIN STORE APPROVAL

Create workflow:

Store Registration

↓

Pending Review

↓

Admin Reviews

↙ ↘

Approve Reject

↓

Published

Admin should be able to see all submitted store information and verification documents.

---

27. COMMISSION SYSTEM

Prepare configurable marketplace commissions.

Example:

Store Order = 100 ₪

Commission = 7%

Platform Fee = 7 ₪

Store Revenue = 93 ₪

Commission should be configurable globally and per store.

---

28. DATABASE MODELS

Create Prisma models for at least:

User

Role

CustomerProfile

Store

StoreOwner

StoreStaff

Driver

Governorate

City

Area

DeliveryZone

Address

Category

Product

ProductImage

Brand

Inventory

Promotion

Cart

StoreCart

CartItem

Order

StoreOrder

OrderItem

Payment

Delivery

Review

Notification

Commission

StoreApplication

AuditLog

Add:

- Primary keys
- Foreign keys
- Indexes
- Created timestamps
- Updated timestamps
- Soft deletion where appropriate

Use UUIDs.

---

29. SECURITY

Implement:

- Password hashing using Argon2 or bcrypt
- JWT authentication
- Refresh tokens
- Role-based authorization
- Input validation
- Rate limiting
- CORS
- Secure HTTP headers
- SQL injection protection through Prisma
- File upload validation
- Maximum file sizes
- Audit logs for sensitive admin operations

Never store plain-text passwords.

Never expose sensitive information through APIs.

---

30. API DESIGN

Create documented endpoints.

Example:

POST /auth/register

POST /auth/login

POST /auth/refresh

GET /governorates

GET /cities

GET /areas

GET /stores

GET /stores/:id

POST /stores/apply

GET /products

GET /products/:id

POST /store/products

PATCH /store/products/:id

GET /cart

POST /cart/items

PATCH /cart/items/:id

DELETE /cart/items/:id

POST /checkout

GET /orders

GET /orders/:id

GET /store/orders

PATCH /store/orders/:id/status

GET /admin/dashboard

GET /admin/stores

PATCH /admin/stores/:id/approve

PATCH /admin/stores/:id/reject

Use DTO validation.

Return consistent API responses and error structures.

---

31. UI / UX DESIGN

The application must look like a modern professional grocery marketplace.

Design principles:

- Arabic RTL first
- Clean interface
- Large product images
- Clear prices
- Prominent Add to Cart button
- Bottom navigation
- Modern cards
- Soft rounded corners
- Accessible typography
- Excellent mobile usability

Bottom navigation:

الرئيسية

المتاجر

السلة

طلباتي

حسابي

Use Palestinian-friendly Arabic terminology.

---

32. CUSTOMER APP SCREENS

Implement:

1. Splash
2. Onboarding
3. Login. (Gmail Account or Phone Number)
4. Registration
5. Location selection
6. Home
7. Search
8. Search results
9. Store list
10. Store details
11. Categories
12. Product details
13. Cart
14. Multi-store cart
15. Checkout
16. Address selection
17. Payment
18. Order confirmation
19. Order tracking
20. Orders
21. Order details
22. Favorites
23. Notifications
24. Profile
25. Addresses
26. Settings

---

33. INITIAL SEED DATA

Create realistic demo data.

Add:

- Palestinian governorates
- Several cities
- Several demo stores
- Categories
- At least 50 grocery products
- Brands
- Prices in ₪
- Inventory
- Promotions
- Demo customers
- Demo store owner
- Demo admin

Use clearly fictional/demo store names and data.

---

34. DEMO ACCOUNTS

Create development-only accounts:

Admin

admin@example.com

Store Owner

store@example.com

Customer

customer@example.com

Use secure development passwords documented only in the README and environment configuration.

Do not use these credentials in production.

---

35. ERROR HANDLING

Create friendly Arabic error messages.

Examples:

المنتج غير متوفر حالياً

الكمية المطلوبة أكبر من الكمية المتوفرة

هذا المتجر مغلق حالياً

لا يمكن إتمام الطلب قبل إضافة عنوان التوصيل

حدث خطأ، يرجى المحاولة مرة أخرى

---

36. EMPTY STATES

Design proper empty states.

Examples:

السلة فارغة

ابدأ بإضافة المنتجات من المتاجر

لا توجد طلبات حتى الآن

لا توجد متاجر متاحة في منطقتك حالياً

لا توجد نتائج للبحث

---

37. RESPONSIVE ADMIN DASHBOARD

Admin dashboard must work on:

- Desktop
- Tablet
- Mobile

Use:

- Sidebar
- Tables

- Cards
- Charts
- Filters
- Pagination
- Search
- Export-ready architecture

---

38. REPORTING

Create reports for:

Platform

- Orders
- Sales
- Commission
- Active customers
- Active stores

Store

- Sales
- Orders
- Best products
- Revenue
- Cancellation rate

Allow filtering:

Today

Yesterday

Last 7 days

Last 30 days

Custom date range

---

39. TESTING

Create automated tests.

Backend:

- Unit tests
- API integration tests
- Authentication tests
- Authorization tests
- Cart tests
- Multi-store checkout tests
- Inventory tests

Critical test:

Customer adds products from Store A

Customer visits Store B

Customer adds products from Store B

Customer returns to Store A

Store A cart is still preserved

Checkout creates separate StoreOrders

This test is mandatory.

---

40. DOCKER

Create Docker configuration for:

PostgreSQL

API

Admin

Provide:

docker-compose.yml

Make local development easy.

---

41. ENVIRONMENT VARIABLES

Create:

DATABASE_URL=

JWT_SECRET=

JWT_REFRESH_SECRET=

STORAGE_PROVIDER=

STORAGE_BUCKET=

STORAGE_ACCESS_KEY=

STORAGE_SECRET_KEY=

PAYMENT_PROVIDER=

PAYMENT_API_KEY=

SMS_PROVIDER=

SMS_API_KEY=

EMAIL_PROVIDER=

EMAIL_API_KEY=

Create .env.example.

Never commit .env.

---

42. README

Create a detailed README containing:

- Project overview
- Architecture
- Requirements
- Installation
- Environment setup
- Database setup
- Prisma migration
- Seed database
- Running API
- Running mobile app
- Running admin dashboard
- Testing
- Docker

- Production deployment
- API documentation

---

43. DEVELOPMENT STRATEGY

Do NOT attempt to build everything as one giant unfinished implementation.

Build in phases.

Phase 1 — Foundation

Implement:

- Repository structure
- PostgreSQL
- Prisma
- Authentication
- User roles
- Geographic system
- Basic API
- Flutter project
- Next.js admin project

Phase 2 — Stores and Products

Implement:

- Store registration
- Store approval
- Categories
- Products
- Inventory
- Store dashboard

Phase 3 — Customer Shopping

Implement:

- Home
- Store discovery
- Store page
- Product details

- Search
- Cart

Phase 4 — Multi-Store Checkout

Implement:

- Multi-store cart
- Address
- Delivery
- Unified checkout
- Master Order
- Store Orders
- Order statuses

Phase 5 — Delivery

Implement:

- Driver accounts
- Assignments
- Delivery statuses
- Tracking architecture

Phase 6 — Advanced Features

Implement:

- Reviews
- Promotions
- Price comparison
- Notifications
- Reports
- AI shopping assistant
- Payment integrations

---

44. IMPORTANT BUSINESS LOGIC

Implement these rules carefully.

Rule 1

A customer can purchase from multiple stores.

Rule 2

Each store receives only its own StoreOrder.

Rule 3

Cancelling one StoreOrder must not automatically cancel other StoreOrders unless the platform explicitly defines that behavior.

Rule 4

Product price must be validated again during checkout.

Rule 5

Inventory must be checked again during checkout.

Rule 6

Store availability must be checked during checkout.

Rule 7

Delivery fee must be calculated per store.

Rule 8

Commission must be calculated per StoreOrder.

Rule 9

The customer should see one unified order experience.

Rule 10

Store owners should never be able to access another store's data.

---

45. FUTURE SCALABILITY

Design the architecture so the platform can later support:

- Pharmacies
- Restaurants
- Bakeries
- Electronics
- Clothing
- Household products
- Local producers
- Olive oil producers
- Wholesale stores

Therefore, do not tightly couple the system to grocery products.

Use a generic marketplace/store/product architecture where possible.

---

46. UI LANGUAGE

The customer-facing application must be Arabic RTL by default.

Examples:

الرئيسية

المتاجر

الأقسام

أضف للسلة

السلة

إتمام الطلب

طلباتي

تتبع الطلب

الحساب

العروض

الأكثر مبيعاً

الأقرب إليك

متاح الآن

مغلق

التوصيل

الإجمالي

Admin and store dashboards should also support Arabic RTL.

---

47. PERFORMANCE

Optimize for users with slower internet connections.

Implement:

- Image compression
- Lazy loading
- Pagination
- API caching where appropriate
- Efficient database queries

- Product image thumbnails
- Infinite scrolling where appropriate

---

48. FINAL ACCEPTANCE CRITERIA

The project is considered successful when this complete scenario works:

1. Customer opens the application.

2. Customer selects:

   Nablus → Nablus City.

3. Application displays available stores.

4. Customer opens Store A.

5. Customer adds:

   Rice

   Oil

   Sugar

6. Customer returns to stores.

7. Customer opens Store B.

8. Customer adds:

   Milk

   Juice

   Water

9. Customer opens Cart.

10. Cart displays:

Store A

3 products

Store B

3 products

11. Customer selects delivery address.

12. Customer selects Cash on Delivery.

13. Customer confirms order.

14. Backend creates:

MasterOrder

├— StoreOrder A

└— StoreOrder B

15. Store A sees only StoreOrder A.

16. Store B sees only StoreOrder B.

17. Customer sees both orders under one checkout.

18. Store A confirms its order.

19. Store B confirms its order.

20. Customer sees status updates independently.

21. Delivery fees are calculated independently.

22. Inventory is reduced correctly.

23. Platform commission is calculated correctly.

24. Admin can see the complete transaction.

This workflow must be fully functional before considering the MVP complete.

---

49. CODING QUALITY REQUIREMENTS

Write production-quality code.

Requirements:

- TypeScript strict mode
- Clean architecture
- Modular services
- Reusable components
- Strong typing
- Input validation
- Proper error handling
- No duplicated business logic
- No hard-coded production secrets
- No fake API responses in completed features
- No placeholder buttons that appear functional but do nothing
- Meaningful comments only where necessary

Before declaring completion:

1. Run the backend.
2. Run database migrations.
3. Run seed.
4. Run tests.
5. Run admin dashboard.

6. Run Flutter application.
7. Test authentication.
8. Test store registration.
9. Test store approval.
10. Test product creation.
11. Test customer shopping.
12. Test multi-store cart.
13. Test checkout.
14. Test StoreOrder isolation.
15. Fix all runtime and compilation errors.

---

50. START NOW

Start by creating the complete repository structure and implementing Phase 1.

Do not merely describe the implementation.

Actually create the files, code, database schema, migrations, seed data, API modules, Flutter application structure, and Next.js admin structure.

After each major phase:

- Run tests.
- Fix errors.
- Verify functionality.
- Update README.
- Continue automatically to the next phase.

Do not stop at a mockup.

The final result must be a functional full-stack MVP that can later be deployed as a real Palestinian grocery marketplace.